

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 4

«Использование технологий вычислений на GPU»

по дисциплине «Языки программирования»

Выполнил
студент гр.5151001/40001

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Семьянов П.В.

Санкт-Петербург
2026г.

1. Цель работы

Изучить методы оптимизации программ с использованием графического процессора (GPU) для повышения производительности вычислений.

2. Ход работы

В качестве вычислительной задачи была выбрана операция двумерной свёртки изображения (2D convolution) с ядром фиксированного размера 15×15 . Данная операция широко применяется в обработке изображений (фильтрация, размытие, повышение резкости, выделение границ) и хорошо подходит для распараллеливания, поскольку вычисление значения каждого пикселя выходного изображения не зависит от других пикселей.

2.1. Алгоритм работы программы

1. Пользователь вводит значение N — размер квадратного изображения ($N \times N$).
2. Если ввод некорректен ($N \leq 0$), программа завершается.
3. Выделяется память под основные массивы:
 - массив `image` размером $N \times N$ элементов (исходное изображение);
 - массив `kernel` размером 15×15 элементов (фильтр свёртки);
 - массив `cpu_out` для хранения результата вычислений
4. Массив изображения заполняется случайными вещественными числами в диапазоне $[0,1]$, имитируя значения яркости пикселей.
5. Массив ядра свёртки также заполняется случайными коэффициентами.
6. Выполняется последовательная (CPU) версия свёртки:
 - для каждого пикселя (x, y) вычисляется сумма произведений соседних пикселей изображения на коэффициенты ядра;
 - учитываются граничные условия (индексы не выходят за пределы изображения);
 - результат записывается в массив `cpu_out`;
 - время выполнения фиксируется с помощью функции `clock()`.
7. Определяется количество доступных OpenCL-платформ с помощью `clGetPlatformIDs`.

8. Из списка платформ выбирается платформа с вендором «AMD» (если она присутствует).
9. Получается список вычислительных устройств (`clGetDeviceIDs`).
 - если доступен GPU — используется он;
 - иначе используется первое доступное устройство (обычно CPU).
10. Создаётся контекст OpenCL (`clCreateContext`) и очередь команд (`clCreateCommandQueue`).
11. Создаётся OpenCL-программа из встроенной строки `kernelSourceCode` (`clCreateProgramWithSource`).
 - каждый рабочий элемент получает свой индекс и проверяет, если индекс вышел за пределы массива, то преждевременно останавливаю программу
 - индекс преобразуется в линейные координаты на изображении
 - двумя циклами программа проходит по блоку 15 на 15 (границы -7 и 7), проверяет границы расположения пикселя, что они в пределах изображения
 - если пиксели в границах изображения, то значение пикселя умножается с значением в фильтре и прибавляется к значению `sum`
 - в получаем изображении записываем в массив по индексу значения перемножения этого пикселя
12. Программа компилируется (`clBuildProgram`); при ошибке выводится лог сборки.
13. Создаётся объект ядра с именем `conv2d` (`clCreateKernel`).

3. Тестирование и результаты работы программы

Размер массива	Время выполнения CPU, с	Время выполнения GPU, с	Ускорение
10	0.00005	0	0
20	0.00012	0	0
40	0.001	0	4.3
80	0.004	0.0001	23.21
160	0.015	0.0001	64.86
320	0.063	0.0002	112.28
640	0.245	0.001	179
1280	1.009	0.010	98.78
2560	3.981	0.040	98.99
5120	15.925	0.110	144.79
10240	64.161	0.351	182.78
20480	255.624	0.648	394.54

Таблица 1. Результаты работы программы

В ходе тестирования было замечено, что при небольших значениях N выполнение на GPU занимало немного больше времени, чем на CPU (разница составляла около 0,001 секунды), что связано с накладными расходами на передачу данных и запуск ядра. Однако при увеличении объёма данных GPU показывал значительно лучшую производительность, обеспечивая ускорение примерно в 4, 23 и до 394 раз по сравнению с CPU.

4. Вывод

В ходе выполнения работы была реализована программа для двумерной свертки изображения с использованием графического процессора. Было изучено применение технологии OpenCL и принципы организации параллельных вычислений на GPU. Реализация показала, что перенос вычислений на видеокарту позволяет существенно повысить производительность при обработке больших объёмов данных. Также была получена практическая задача работы с OpenCL, загрузкой и компиляцией ядра, управлением памятью и синхронизацией вычислений.

Приложения

main.c

```
#include <CL/cl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define K 15
#define R (K / 2)

const char* kernelSourceCode =
    "__kernel void conv2d(__global const float* input,\n"
    "                      __global const float* filter,\n"
    "                      __global float* output,\n"
    "                      const int N)\n"
    "{\n"
    "    int idx = get_global_id(0);\n"
    "    int total = N*N;\n"
    "    if (idx >= total) return;\n"
    "\n"
    "    int x = idx % N;\n"
    "    int y = idx / N;\n"
    "\n"
    "    float sum = 0.0f;\n"
    "    for(int ky=-7; ky<=7; ky++){ \n"
    "        for(int kx=-7; kx<=7; kx++){ \n"
    "            int nx = x + kx;\n"
    "            int ny = y + ky;\n"
    "            if(nx>=0 && nx<N && ny>=0 && ny<N){ \n"
    "                float v = input[ny*N + nx];\n"
    "                float w = filter[(ky+7)*15 + (kx+7)];\n"
    "                sum += v*w;\n"
    "            } \n"
    "        } \n"
    "    } \n"
    "    output[idx] = sum;\n"
    "}";
```

```

int main()
{
    int N;
    printf("Enter N (image NxN): ");
    if (scanf("%d", &N) != 1 || N <= 0)
        return 0;

    int total = N * N;

    float* image = (float*)malloc(total * sizeof(float));
    float* kernel = (float*)malloc(K * K * sizeof(float));
    float* cpu_out = (float*)malloc(total * sizeof(float));

    for (int i = 0; i < total; i++)
        image[i] = (float)rand() / RAND_MAX;

    for (int i = 0; i < K * K; i++)
        kernel[i] = (float)rand() / RAND_MAX;

    /* ===== CPU ===== */
    clock_t t0 = clock();

    for (int y = 0; y < N; y++)
        for (int x = 0; x < N; x++)
        {
            float sum = 0.0f;
            for (int ky = -R; ky <= R; ky++)
                for (int kx = -R; kx <= R; kx++)
                {
                    int nx = x + kx, ny = y + ky;
                    if (nx >= 0 && nx < N && ny >= 0 && ny < N)
                    {
                        float v = image[ny * N + nx];
                        float w = kernel[(ky + R) * K + (kx + R)];
                        sum += v * w;
                    }
                }
            cpu_out[y * N + x] = sum;
        }
}

```

```

double cpu_time = (double)(clock() - t0) / CLOCKS_PER_SEC;

cl_uint numPlatforms = 0;
clGetPlatformIDs(0, NULL, &numPlatforms);

cl_platform_id* platforms =
    (cl_platform_id*)malloc(numPlatforms *
sizeof(cl_platform_id));
clGetPlatformIDs(numPlatforms, platforms, NULL);

int selected = 0;
for (int i = 0; i < numPlatforms; i++)
{
    char buf[100];
    clGetPlatformInfo(platforms[i], CL_PLATFORM_VENDOR,
sizeof(buf), buf,
NULL);
    if (strstr(buf, "AMD") || strstr(buf, "Advanced Micro
Devices"))
    {
        selected = i;
        break;
    }
}

cl_uint numDevices = 0;
clGetDeviceIDs(platforms[selected], CL_DEVICE_TYPE_ALL, 0, NULL,
&numDevices);

cl_device_id* devices =
    (cl_device_id*)malloc(numDevices * sizeof(cl_device_id));
clGetDeviceIDs(platforms[selected], CL_DEVICE_TYPE_ALL,
numDevices, devices,
NULL);

cl_device_id device = NULL;
for (cl_uint i = 0; i < numDevices; i++)
{
    cl_device_type type;
    clGetDeviceInfo(devices[i], CL_DEVICE_TYPE, sizeof(type),

```



```

&type, NULL);
    if (type == CL_DEVICE_TYPE_GPU)
    {
        device = devices[i];
        printf("GPU selected\n");
        break;
    }
}
if (!device)
{
    device = devices[0];
    printf("CPU selected\n");
}

cl_int status;
cl_context context = clCreateContext(NULL, 1, &device, NULL,
NULL, &status);

cl_command_queue queue = clCreateCommandQueue(context, device,
0, &status);

cl_program program =
    clCreateProgramWithSource(context, 1, &kernelSourceCode, NULL,
&status);

status = clBuildProgram(program, 1, &device, NULL, NULL, NULL);

if (status != CL_SUCCESS)
{
    size_t logSize;
    clGetProgramBuildInfo(program, device, CL_PROGRAM_BUILD_LOG,
0, NULL,
        &logSize);
    char* log = (char*)malloc(logSize);
    clGetProgramBuildInfo(program, device, CL_PROGRAM_BUILD_LOG,
logSize,
        log, NULL);
    printf("Build error:\n%s\n", log);
    return 0;
}

```

```

cl_kernel k = clCreateKernel(program, "conv2d", &status);

clock_t tg = clock();

cl_mem d_image =
    clCreateBuffer(context, CL_MEM_READ_ONLY |
CL_MEM_COPY_HOST_PTR,
        total * sizeof(float), image, &status);

cl_mem d_kernel =
    clCreateBuffer(context, CL_MEM_READ_ONLY |
CL_MEM_COPY_HOST_PTR,
        K * K * sizeof(float), kernel, &status);

cl_mem d_out = clCreateBuffer(context, CL_MEM_WRITE_ONLY,
        total * sizeof(float), NULL, &status);

clSetKernelArg(k, 0, sizeof(cl_mem), &d_image);
clSetKernelArg(k, 1, sizeof(cl_mem), &d_kernel);
clSetKernelArg(k, 2, sizeof(cl_mem), &d_out);
clSetKernelArg(k, 3, sizeof(int), &N);

size_t globalWorkSize = total;
clEnqueueNDRangeKernel(queue, k, 1, NULL, &globalWorkSize, NULL,
0, NULL,
        NULL);
clFinish(queue);

float* gpu_out = (float*)malloc(total * sizeof(float));
clEnqueueReadBuffer(queue, d_out, CL_TRUE, 0, total *
sizeof(float),
        gpu_out, 0, NULL, NULL);

double gpu_time = (double)(clock() - tg) / CLOCKS_PER_SEC;

/* ===== checksum ===== */

double s1 = 0, s2 = 0;
for (int i = 0; i < total; i++)

```

```
{
    s1 += cpu_out[i];
    s2 += gpu_out[i];
}

printf("checksum CPU: %.6f\n", s1);
printf("checksum GPU: %.6f\n", s2);
printf("time CPU: %.3f sec\n", cpu_time);
printf("time GPU: %.3f sec\n", gpu_time);
printf("speedup: %.2fx\n", cpu_time / gpu_time);

return 0;
}
```